

Knowledge Graph Docs

Food Fraud Lifecycle Ontology (FFLO) — Knowledge-Graph Setup

This document walks through how the food-safety knowledge graph is set up, from the ontology design down to the exported RDF. It is organised into the four stages of the pipeline: **entity and relation type design**, **entity and relation extraction**, **NLI verification**, and **KG construction**. The implementation lives in `src/extraction/`, `src/validation/`, and `src/kg/`; the website-facing exports are produced by `scripts/export_kg_data.py`. Evaluation measures and a short note on the ID namespace are collected in the appendices.

Overview

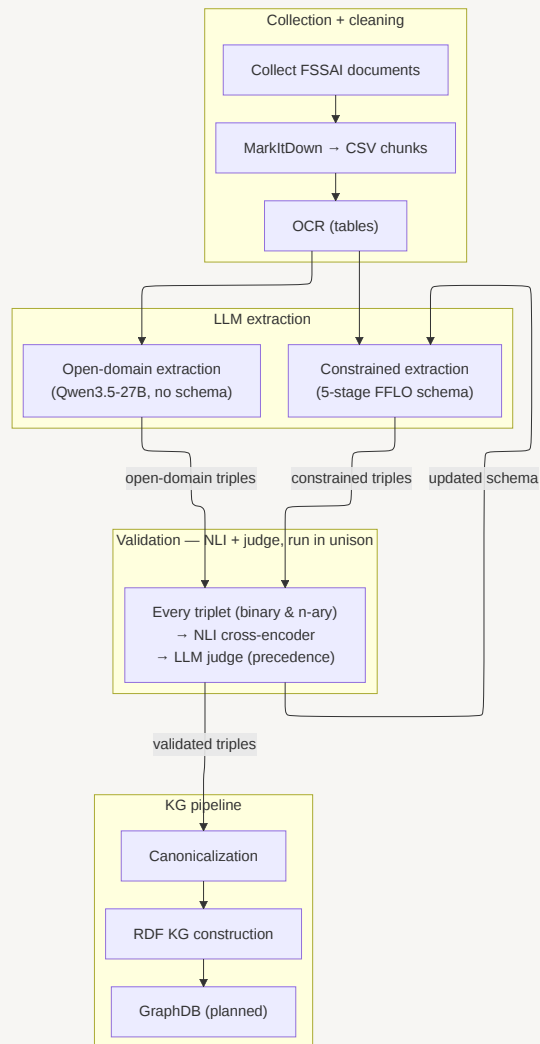
We extract structured `(subject, predicate, object)` triplets from the FSSAI regulatory corpus in two parallel runs:

- a **constrained** run, guided by the FFLO ontology schema (5 stages), and
- an **open-domain** run with no schema (Qwen3.5-27B), used to discover what the schema misses.

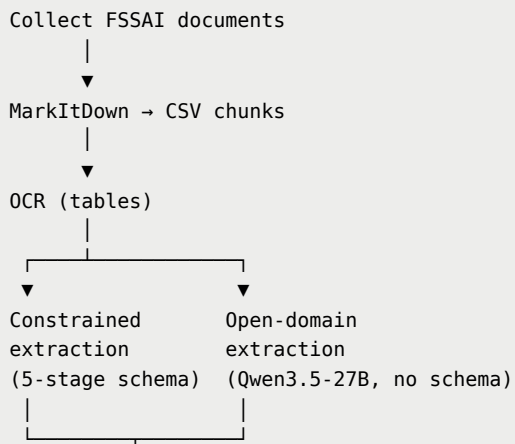
Each extracted fact is either a **binary relation** (a single subject–predicate–object assertion) or an **n-ary event** (a fact with several linked attributes, reified as multiple triplets sharing one synthetic event id). Every triplet — binary or n-ary — is then validated against its source evidence by a combined **NLI + LLM-judge** layer that runs **both** checks in unison (the judge taking precedence). Validated triples are merged, canonicalised, and exported as a knowledge graph; the open-domain run feeds its discovered gaps back into the schema for the constrained run.

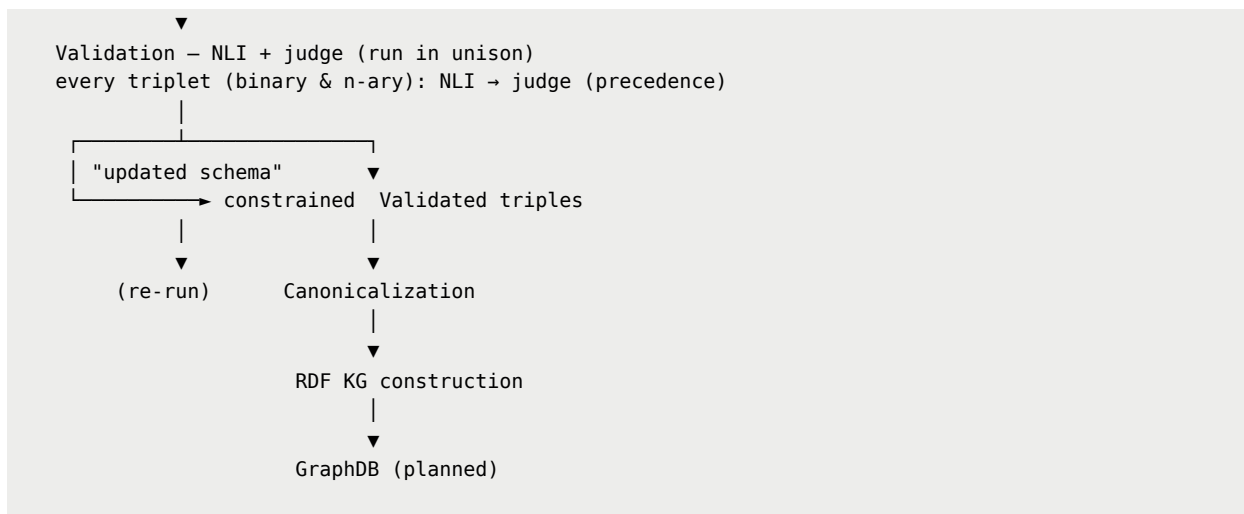
This constrained/open-domain split, canonicalised afterward, follows the same general pattern as the Extract–Define–Canonicalize framework for LLM-based KG construction [3], applied here on top of the FKG.in Indian-food ontology [1] (cf. FoodKG [2] as a non-regional precedent for food-domain KG construction).

System diagram



ASCII fallback:





1. Entity and Relation Type Design

The FFLO ontology organises food-fraud knowledge into five lifecycle stages and is defined in two places: the human-readable reference `src/schema_iterations/ schema_v3.md` (the full entity/relation catalogue) and the machine-readable `src/extraction/schema_config.json` (the same types, relations, domain/range constraints and subclass hierarchy that the extraction, validation and RDF steps all read). The current schema has **89 entity types, 72 relations, 45 subclass-of axioms** and **72 domain/range entries**.

1.1 The five-stage lifecycle

Entities and relations are grouped by the stage of the fraud lifecycle they describe:

Stage	What it models	Core entities
ACT	who commits the fraud, with what, and how	<code>Adulterant</code> , <code>AdulterationAct</code> , <code>AdulterationMethod</code> , <code>FoodBusinessOperator</code> , <code>FoodAdditive</code> , <code>FraudType</code>
SPREAD	how the adulterated product moves through the supply chain	<code>SupplyChainStep</code> (Production / Processing / Storage / Distribution / Retail / Import), <code>SpreadEvent</code> , <code>TransformationEvent</code>
DETECTION	how the fraud is found and measured	<code>IncidentFinding</code> , <code>Sample</code> , <code>Analysis</code> , <code>AnalysisResult</code> , <code>DetectionMethod</code> (LaboratoryTest / FieldTest / SensoryTest), <code>Measurement</code> , <code>Property</code>
REGULATION	the norms, limits and enforcement actions	<code>RegulatoryDocument</code> , <code>FoodStandard</code> , <code>PermissibleLimit</code> , <code>FoodCategory</code> , <code>RegulatoryBody</code> , <code>RegulatoryAction</code> , <code>Violation</code>

Stage	What it models	Core entities
HEALTH	the consequence of the fraud	<code>HealthEffect</code> , <code>CausalHealthEffect</code> , <code>AcuteEffect</code> , <code>ChronicEffect</code> , <code>VulnerablePopulation</code>

1.2 Entity types and subtypes

Types carry a `subclass_of` hierarchy, declared in `schema_config.json` . The key hierarchies are:

- **Adulterant** is split by origin into `SyntheticAdulterant` (industrial chemical, e.g. Sudan dye, melamine), `NaturalAdulterant` (cheaper natural substitute, e.g. water in milk), and `ContaminantAdulterant` (present above limits without being added, e.g. pesticide residue), with `Microorganism` as a subtype of `ContaminantAdulterant` (Salmonella, E. coli).
- **FoodAdditive** (a *permitted* substance, distinct from `Adulterant`) and its sibling `ProcessingAid` are both `fkg:ChemicalIngredient` .
- **SupplyChainStep** subsumes `Production` , `Processing` , `Storage` , `Distribution` , `Retail` and `Import` .
- **IncidentFinding** (the core detection entity: “adulterant X found in food Y”) is specialised into `LabConfirmed` , `FieldDetected` , `SurveyAggregated` and `RecallTriggered` .
- **DetectionMethod** is specialised into `LaboratoryTest` , `FieldTest` and `SensoryTest` .
- **HealthEffect** is specialised into `CausalHealthEffect` , `AcuteEffect` and `ChronicEffect` .

1.3 Controlled vocabularies

Several entity types are closed vocabularies rather than free text:

Type	Allowed values
<code>fflo:FraudType</code>	Dilution / Substitution / Mislabeling / Masking / Contamination
<code>fflo:IntentionalityLevel</code>	Intentional / Negligent / Incidental
<code>fflo:AdditiveFunction</code>	Preservative, Antioxidant, Emulsifier, Stabilizer, AcidityRegulator, FlourTreatmentAgent, Sequestrant, HumectantAdditive, Colourant, SweeteningAgent
<code>fflo:ObligationType</code>	LabelDeclaration, MarkAffixation, SignatureRequirement, PackagingRequirement, FormFiling, TestingRequirement, ConformanceRequirement, ProductionRequirement

Type	Allowed values
<code>fflo:RegulatoryActionType</code>	Seizure, Removal, PenaltyReduction, Authorization, Sealing

1.4 Relations and domain/range

Each relation declares a **domain** (the type of its subject) and a **range** (the type of its object). Domain/range constraints are what the schema validator and the judge's `schema_invalid` branch check against. Representative examples:

Relation	Domain	Range	Meaning
<code>fflo:hasAdulterant</code>	<code>fkg:Food</code>	<code>fflo:Adulterant</code>	food is commonly adulterated with this substance
<code>fflo:hasAdulterationMethod</code>	<code>fkg:Food</code>	<code>fflo:AdulterationMethod</code>	how the food is typically adulterated
<code>fflo:identifies</code>	<code>fflo:IncidentFinding</code>	<code>fflo:Adulterant</code>	what the finding found
<code>fflo:foundIn</code>	<code>fflo:IncidentFinding</code>	<code>fkg:Food</code>	which food it was found in
<code>fflo:compliesWith</code>	<code>fkg:Food</code> / <code>FoodBusinessOperator</code>	<code>fflo:FoodStandard</code> / <code>RegulatoryDocument</code>	compliance obligation
<code>fflo:hasDefinition</code>	<code>fflo:FoodCategory</code> / <code>Adulterant</code>	<code>xsd:string</code>	verbatim FSSAI definition
<code>fflo:causesEffect</code>	<code>fflo:Adulterant</code>	<code>fflo:HealthEffect</code>	adulterant's health consequence

The full domain/range table lives in `schema_v3.md` (one section per stage) and in `schema_config.json`. Only relations declared there may be extracted or emitted to RDF.

1.5 The `PermissibleLimit` decomposition pattern

A numeric limit is never meant to be a single bare relation. The extractor decomposes it into a fixed three-relation pattern that grounds the limit to its property, value and food:

```
PermissibleLimit →forProperty→ ssn:Property
PermissibleLimit →hasValue→ fso:Measurement
```

PermissibleLimit →inFood→ fkg:Food

This single pattern absorbs the many surface forms the text uses (`hasAcidityLimit`, `hasResidueLimit`, `hasMaxFatContent`, ...) — the property name (acidity, residue, fat content) becomes the `ssn:Property` node, the quantity the `fso:Measurement` node, and the food the `fkg:Food` node. This is a canonicalisation rule, not a schema gap.

1.6 Schema evolution

The schema is not frozen. The open-domain run (§2) surfaces concepts the schema did not name, and the entailed subset is folded back in as an updated schema (`89 entity types / 72 relations`). This is how `ObligationType`, `RegulatoryOfficer`, `Microorganism`, `SamplingPlan` and `RegulatoryActionType` (and the relations `hasObligation`, `hasScientificName`, `governsSampling`, `isEmpoweredTo`, `actsOnBehalfOf`) entered the ontology.

2. Entity and Relation Extraction

The model is prompted with the FFLO ontology (entity types, relations, and their domain/range constraints) and extracts triplets using **only** those names.

Three behaviours follow directly from the prompt:

1. **Limit decomposition** — a numeric limit is decomposed into the fixed three-relation pattern of §1.5 rather than a single bare relation.
2. **Event reification** — n-ary facts (an event with multiple attributes, e.g. an adulterant *and* a detection method *and* a location) are decomposed into several triplets sharing one synthetic `event_id`.
3. **UNMAPPED** — a fact that cannot be expressed with the current schema is emitted with predicate `UNMAPPED` (and a `mismatch_note`) rather than being forced into an existing relation.

Example (chunk `src_pdf_001_chunk_0005`, evidence “claims on source of dietary fibre”): one limit event becomes three triplets sharing `ev_dietary_fibre_claim_0001`:

subject	predicate	object	confidence
<code>ev_dietary_fibre_claim_0001</code> (<code>PermissibleLimit</code>)	<code>fflo:forProperty</code>	<code>Dietary Fibre</code> (<code>ssn:Property</code>)	0.95
<code>ev_dietary_fibre_claim_0001</code>	<code>fflo:hasValue</code>	<code>3g/100g</code> (<code>fso:Measurement</code>)	0.95

subject	predicate	object	confidence
ev_dietary_fibre_claim_0001	fflo:inFood	food products (fkg:Food)	0.90

A fact the schema cannot express is emitted with predicate `UNMAPPED` (16 rows in the initial run), e.g. an amendment rule “substitution of ‘five years’ with ‘eight years’” on a `RegulatoryDocument`.

Two extraction modes are used:

Mode	Prompt	Model	Purpose
Constrained (schema-guided)	<code>prompts/triplet_extraction.txt</code>	Qwen	production triplets
Open-domain (unconstrained)	<code>prompts/unconstrained_extraction.txt</code>	Qwen3.5-27B	discover schema gaps

Open-domain (unconstrained) run

The open-domain run extracts from the same chunks with **no schema**: the model freely names entity types and relations from the observed text (e.g.

`hasMinSaponificationValue`, `includesSubcategory`). It produced **12,434** free-form triplets. The free-form output is then post-processed in a *flag* pass: entity types and relation names are normalised (string heuristics plus embedding clustering), every triplet is checked by an NLI cross-encoder — with negation handled by rephrasing the hypothesis as “It is NOT the case that ...” — and the normalised vocabulary is compared against the FFLO schema to separate covered concepts from candidate new ones. Finally a combined NLI + LLM-judge pass (`nli_llm_judge.py`: chunk-level and span-level NLI scores plus a Qwen judge, with the judge taking precedence; agreed → accepted, disagreed → contested) keeps only the entailed relations: **5,882** of the 12,434 free-form triplets.

The purpose of this run is *coverage* rather than *production*: the entailed free-form triplets surface relation and type concepts the schema did not name, and those gaps become the updated schema of §1.6.

Open-domain relation canonicalisation

Because the open-domain run names relations freely, its free-form predicates must be canonicalised into FFLO relation names before they can drive schema updates. This is a separate, **LLM-grounded** step

(`src/validation/llm_canonicalise.py`), distinct from the row-level dedup and entity-ID minting applied to the constrained run. It reads the

similarity-suggested clusters and, for each, pulls grounding examples (subject/object type pairs plus evidence spans) and asks Qwen to **verify each cluster before naming it** — splitting clusters that mix distinct concepts, rejecting out-of-scope/bureaucratic relations, and flagging ambiguous ones for review — with an antonym guard (`include / exclude` , `max / min` , ...) and a confidence threshold. Output is `canonical_map.json` (auto-applied merges) plus `canonical_map_needs_review.json` and `canonical_map_out_of_scope.json` for the rest.

3. NLI Verification

Every triplet — from the constrained run and the open-domain run alike, binary and n-ary — is passed through **both** an NLI cross-encoder and an LLM judge, run in unison. The NLI cross-encoder provides a fast entailment signal by verbalising the triplet into a single premise–hypothesis pair; the LLM judge provides the authoritative verdict, reading the full evidence span and checking each argument slot individually and jointly (plus schema-level corrections). When the two disagree, the **judge takes precedence**; agreement is recorded as `agreed_all` , `agreed_partial` , or `contested` . This dual-validator design is consistent with recent automatic KG-validation frameworks that combine entailment scoring with LLM judges [4, 5], and with curator-style LLM validation of triple insertions against a target schema [8].

Synthetic IDs and verbalisation

NLI models need a single natural-language hypothesis per triplet, but many entities are **synthetic** — pipeline-generated names that never appear verbatim in the text (`ev_dietary_fibre_claim_0001` , `DeclarePotassium` , `Measurement_50_mg_kg`). Before inference, the pipeline decides which entities are synthetic (a name that is not a substring of its evidence span) and assigns each a deterministic hash ID — `sha256(snippet · name · type)[:12]` — so the same synthetic node resolves consistently across all triplets in a chunk (`src/validation/synthetic_ids.py`). Resolution order: CSV `subject_id` / `object_id` columns first, then a gold annotation file, then the heuristic. Each synthetic ID is given a descriptive label from its entity type (`type_phrase` , e.g. “the permissible limit”) and the triplet is verbalised into a sentence using a relation template (`verbalise.py`). A small sample of the templates:

predicate	verbalised hypothesis ({subj} / {obj} are placeholders)
fflo:belongsToCategory	{subj} belongs to the food category {obj}
fflo:hasAdulterant	{subj} has the adulterant {obj}
fkg:hasIngredient	{subj} contains the ingredient {obj}
fflo:hasDefinition	{subj} is defined as {obj}
fflo:hasPermissibleLimit	{subj} has a permissible limit of {obj}
fflo:compliesWith	{subj} complies with {obj}

The cross-encoder (`cross-encoder/nli-deberta-v3-small`) then scores the pair (`premise, hypothesis`) — the premise is the evidence span (or the full chunk) — and returns one of three labels with a softmax confidence: **entailment**, **neutral**, or **contradiction**. These map onto the judge-style verdicts (entailment → `entailed`, neutral → `uncertain`, contradiction → `not_entailed`), but the NLI label is only a fast signal; the judge's read of the full span is what decides.

Arity and why the judge is needed

Facts differ in **arity**: a binary relation is a single assertion, while an n-ary event bundles several linked arguments. NLI cross-encoders are fine-tuned on binary premise–hypothesis pairs, so a multi-argument relation can only be handled by decomposing it into separate checks (losing the joint constraint that all arguments hold together) or collapsing it into one compound hypothesis (out of distribution). The LLM judge reads the full evidence span and checks each slot individually and jointly, which is exactly what n-ary content requires. Because distinguishing arity automatically is unreliable, in practice **every triplet — binary and n-ary alike — is passed through both validators**, with the judge authoritative.

Worked examples

A triplet is validated in three steps: verbalise it into a hypothesis, score the hypothesis against its evidence with the cross-encoder, then have the judge read the full evidence span.

Binary example. The evidence span *“They shall conform to the provisions mentioned under sub-regulation 2.2.1”* yields the triplet:

```

Subject   : Raw edible oils           (fkg:Food)
Predicate : fflo:compliesWith
Object    : sub-regulation 2.2.1      (fflo:RegulatoryDocument)

```

Verbalised hypothesis (via the `compliesWith` template): *“Raw edible oils complies with sub-regulation 2.2.1.”* The cross-encoder scores the pair (premise = evidence span, hypothesis) and returns **entailment** (confidence ≈ 0.94 over contradiction / entailment / neutral), which maps to `entailed`. The judge reads the same span and agrees, so the fact is recorded `agreed_all` and kept.

N-ary example. The claim *“a dietary-fibre source claim requires at least 3 g/100 g”* is one event `ev_dietary_fibre_claim_0001` reified into three linked triplets. Each slot is verbalised and scored independently for NLI, but the authoritative check is the judge's joint review of the whole event:

triplet	verbalised hypothesis	NLI	judge verdict
<code>ev_dietary_fibre_claim_0001</code> → <code>forProperty</code> → <code>Dietary Fibre</code>	“the permissible limit is for the property Dietary Fibre”	entailment	partially_entailed (0.85)
<code>ev_dietary_fibre_claim_0001</code> → <code>hasValue</code> → <code>3g/100g</code>	“the permissible limit has a value of 3g/100g”	entailment	partially_entailed (0.85)
<code>ev_dietary_fibre_claim_0001</code> → <code>inFood</code> → <code>food products</code>	“the permissible limit applies in the food food products”	neutral	partially_entailed (0.85)

The judge's rationale is slot-specific — *“Text states a minimum threshold for a claim, not a fixed permissible limit value”* — which a binary-only NLI check misses because it cannot see the three slots jointly.

Negation. When the source states that something does *not* hold, the hypothesis is rephrased before scoring so the cross-encoder reads the negated meaning: a fact extracted from a negative sentence becomes *“It is NOT the case that {subj} ...”*.

Chunk vs span premise. The same hypothesis scores differently depending on what premise it is checked against. On the open-domain run, scoring against the full chunk gave entailment 4,147 / contradiction 1,304 / neutral 431, whereas scoring against the narrower evidence span gave entailment 4,495 / contradiction 429 / neutral 958. The premise choice materially changes the NLI label, which is why the judge — which reads the span itself — is authoritative.

Two more NLI examples

Two real examples from the open-domain run, where both validators score every triplet, show the two agreement outcomes.

Agreement (`agreed_all`).

```

subject : Natural mineral waters and source waters (FoodCategory)
predicate : isCharacterizedBy
object : presence of certain mineral salts (Property)
evidence : "characterized by the presence of certain mineral salts"
verbalised: "Natural mineral waters and source waters isCharacterizedBy presence of certain mineral salts"

```

The cross-encoder returns **entailment** against both premises (chunk 0.78, span 0.64); the judge confirms **entailed** (0.98) and the two agree (`agreed_all`).

Partial agreement (`agreed_partial`) — the premise swings the NLI label.

```

subject : Non-alcoholic beverages (FoodCategory)
predicate : includesSubcategory
object : fruit and vegetable juices (FoodCategory)
evidence : "includes ... fruit and vegetable juices (14.1.2)"
verbalised: "Non-alcoholic beverages includesSubcategory fruit and vegetable juices"

```

The same hypothesis scores **contradiction** against the full chunk (0.54) but **entailment** against the narrower evidence span (0.41). The judge, reading the span, returns **entailed** (0.98) and overrides the chunk-level NLI (`agreed_partial`) — the premise-choice effect made concrete.

The judge's four branches

Branch	Trigger	Verdicts
<code>zero_triplet</code>	chunk produced no triplets	<code>correct_zero</code> , <code>missed_relation</code>
<code>schema_mismatch</code>	relation outside schema	<code>needs_schema_extension</code> , <code>maps_to_existing</code> , <code>unsupported</code>
<code>schema_invalid</code>	domain/range/type failure	<code>entailed_wrong_relation</code> , <code>entailed_wrong_types</code> , <code>entailed_direction_reversed</code> , <code>not_entailed</code>
<code>schema_valid</code>	passed schema checks	<code>entailed</code> , <code>partially_entailed</code> , <code>not_entailed</code>

End-to-end worked example

Tracing one fact through the whole pipeline makes the process concrete. The chunk `src_pdf_001_chunk_0008` contains the sentence *"salt substitute, composed entirely or partially of a potassium salt"*.

- 1. Extraction (§2).** The constrained run emits the triplet `salt substitute (fkg:Food) → fkg:hasIngredient → potassium salt (fkg:ChemicalIngredient)`

with confidence 0.95.

2. **Verbalisation (§3).** The `hasIngredient` template renders it as “*salt substitute contains the ingredient potassium salt*”; the evidence span becomes the NLI premise.
3. **NLI + judge (§3).** The cross-encoder scores the pair as entailment; the judge reads the span and returns `entailed`. The two agree (`agreed_all`).
4. **Construction (§4).** After canonicalisation, `salt substitute` gets ID `fkg:F00_00001` and `potassium salt` gets `fkg:CHE_00001`. The RDF builder emits (with `fkg:` / `rdfs:` / `fflo:` prefixes):

```
<urn:fflo:resource:foo-00001> a fkg:Food ; rdfs:label "salt substitute" .  
<urn:fflo:resource:foo-00001> fkg:hasIngredient <urn:fflo:resource:che-00001> .  
<urn:fflo:resource:che-00001> a fkg:ChemicalIngredient ; rdfs:label "potassium salt" .
```

plus an `fflo:Assertion` node that reifies the statement with its `confidence`, `evidenceSpan`, `snippetId` and `sourceFile`, so the triple can always be traced back to the sentence it came from.

4. KG Construction

4.1 Dedup — three-phase row-level canonicalisation

Before entities are assigned IDs, the verified set is cleaned and deduplicated (`src/extraction/canonicalise_kg.py`). The 13,355 verified triplets are first cleaned: 30 rows with no source metadata (empty `source_id` / `source_file` / `source_type` and a malformed `snippet_id`, spread over four chunks: `src_pdf_0232` 15, `src_pdf_0234` 8, `src_pdf_0236` 6, `src_pdf_0039` 1) are dropped, leaving 13,325 rows.

Because chunk overlap means the same fact and the same entity are often extracted more than once with small surface-form variations, the remaining rows are reduced in three phases:

1. **Surface-form merge** — merge duplicate entity mentions via fuzzy matching and a graph-neighbour check, guarded against chemical-type collisions and **event-ID collisions** (73 fuzzy merges; 206 chemical, 673 neighbour and 9,389 event cases blocked).
2. **Type-conflict resolution** — resolve genuine type conflicts versus mistyped noise per entity (8,767 resolved; 1,027 genuine conflicts left for review).

3. **Row dedup** — drop near-identical triples via exact (subject, predicate, object) matching plus a Jaccard-based evidence-span pass, taking **13,325** → **12,400** rows (925 dropped). When exact-duplicate rows from distinct chunks are collapsed, the surviving row keeps its own `snippet_id` and records the additional snippets in a new `alt_snippet_ids` column rather than mutating `snippet_id` (which would break the join back to `chunks.csv`).

4.2 Entity-ID minting and event-ID qualification

After dedup, entities are canonicalised from raw text spans into shared graph IDs (`fkkg:<TYPE>_NNNNN`) via type-blocking, normalisation, and char n-gram TF-IDF similarity clustering (≥ 0.90 auto-merge, 0.75–0.90 flagged for review). Similarity-based auto-merging is disabled for the reasons noted below; only exact-normalised-string matches merge automatically, with a cross-type exact-match pass and guards against conflicting numeric values and mismatched compound qualifiers.

Event-reification IDs (`ev_*`) are treated specially. The extractor reuses the same `ev_*` name across chunks (and sources) for distinct events, so these names are not globally unique. The canonicalisation step therefore **qualifies** each event ID with its source number and chunk index (`ev_dietary_fibre_claim_0001` → `ev_dietary_fibre_claim_0001_001_5` , i.e. `<name>_<source>_<chunk>`) and **excludes event IDs from fuzzy merging**, so distinct events never collapse into one node.

Two audits informed this design. An audit found 70 cases where identical entity strings had been split across multiple IDs (inconsistent type labels and incomplete merge propagation), fixed with a deterministic remap. A broader near-duplicate scan (cross-type, similarity ≥ 0.70) returned 6,151 pairs, but most were false positives from event-IDs, numeric/measurement strings, and values differing only by an embedded number; high similarity alone was unreliable even at ≥ 0.90 (e.g. "Schedule I" vs "II", "Ethyl" vs "Methyl").

The final run over the deduplicated verified set assigns **7,990** canonical entity IDs (1,004 cross-type exact-match merges; 1,808 borderline pairs flagged for review), populating `subject_id` / `object_id` . Together with the qualified event IDs (about 1,737) the CSV holds 9,727 distinct IDs; the RDF builder emits 8,703 of them as resource IRIs (§4.3).

4.3 RDF construction

The RDF graph is built from the canonicalised CSV by `src/kg/build_rdf_kg.py` . Canonical IDs map to `urn:fflo:resource:<TYPE>_<NNNNN>` instance IRIs (8,703

distinct entities; event IDs are slugified to hyphens, e.g.

`ev-limit-aluminium-0009-008-32`); the ontology header, class/subclass axioms and property declarations are written to `ontology.ttl`, the data to `kg.ttl` / `kg.nt`, and a self-contained merge to `kg_full.ttl`. The builder asserts a single `rdf:type` per node — a node with an unresolved type conflict keeps only its first-seen type, and the suppressed conflicts (2,792 in this run) are recorded in `build_report.json` rather than emitted as multiple `rdf:type` statements. Rows that drift from the schema are quarantined to `schema_drift.csv`.

Build totals (`build_report.json`): 12,400 input rows → **12,343 accepted** and **57 quarantined**; 12,334 distinct statement triples (9 duplicates); 8,703 entities; 165,577 RDF triples overall. Quarantine reasons (a row can hit more than one): `unknown_predicate` 24, `unknown_object_type` 20, `object_value_kind_mismatch` 11, `unmapped_predicate` 7.

Provenance layer. Each accepted row is also written as an `fflo:Assertion` node (`urn:fflo:assertion:<hash>`, 12,343 in `kg.ttl`) that reifies the statement and carries its evidence: `assertsSubject`, `assertsPredicate`, `assertsObject`, `confidence`, `evidenceSpan`, `snippetId`, `sourceFile`, `sourceId`, `sourceType` and `chunkIndex`. Any triple can therefore be traced back to the chunk and text span it came from.

Known limitations of the current build (`kg.ttl`):

- `PermissibleLimit` nodes are not always fully decomposed (§1.5). Of 1,261 `ev-limit-*` nodes, 533 have all of `hasValue`, `inFood` and `forProperty` (hasValue 1,046, inFood 934, forProperty 823); 223 nodes use `appliesTo` instead of `inFood`.
- 10 `PermissibleLimit` nodes (7 of them `ev-limit-*`) still carry more than one value in a slot (e.g. two `inFood` foods).
- Resolved in this build: no `(+n)` suffixes in `snippetId`, no assertions with empty source fields, and no node with more than one `rdf:type`.

4.4 GraphDB loading (planned)

Loading the exported RDF into GraphDB is the remaining construction step; it is planned rather than live.

Appendix A — Evaluation measures

A.1 Overall pipeline yield

Stage	Count
Extracted (initial)	15,501
of which event-linked (n-ary)	5,420
Verified (post-judge)	13,355
kept unchanged	11,476
corrected (predicate/type/direction fixed)	1,879
dropped	2,146

Reconciliation: 11,476 kept + 1,879 corrected = 13,355 verified; 15,501 – 13,355 = 2,146 dropped.

A.2 Judge verdict distribution

Verdict	Count
entailed	4,930
partially_entailed	2,151
not_entailed	2,507
entailed_wrong_relation	2,315
entailed_wrong_types	3,014
entailed_direction_reversed	552
correct_zero	47
judge_error	8

The table sums to **15,524**: 15,516 successful verdicts plus the 8 `judge_error` rows.

A.3 Judge support-rate by relation type

The support rate is the fraction of *supported* verdicts (`entailed` , `partially_entailed` , `entailed_wrong_relation` , `entailed_wrong_types` , `entailed_direction_reversed`) over all non-error verdicts.

predicate	judged	supported	not supported	support rate
fkg:hasIngredient	1,908	1,525	383	79.9%
fflo:hasFunction	1,851	1,679	172	90.7%
fflo:hasValue	1,752	1,385	367	79.1%
fflo:forProperty	1,690	1,489	201	88.1%
fflo:inFood	1,648	1,202	446	72.9%

predicate	judged	supported	not supported	support rate
<code>fflo:hasDefinition</code>	1,072	1,016	56	94.8%
<code>fflo:hasObligation</code>	1,035	884	151	85.4%
<code>fflo:hasSubcategory</code>	765	689	76	90.1%
<code>fflo:appliesTo</code>	668	607	61	90.9%
<code>fflo:hasScientificName</code>	329	178	151	54.1%
<code>fflo:isEmpoweredTo</code>	314	272	42	86.6%
<code>fflo:hasPermissibleLimit</code>	314	230	84	73.2%
<code>fflo:belongsToCategory</code>	302	243	59	80.5%
<code>fflo:compliesWith</code>	258	250	8	96.9%
<code>fflo:hasAdulterant</code>	144	41	103	28.5%
<code>fflo:hasAdulterationMethod</code>	72	25	47	34.7%

Low-support relations (`hasAdulterant` 28.5%, `hasAdulterationMethod` 34.7%, `hasScientificName` 54.1%) are the ones most often extracted from hedged or procedural text and are the clearest candidates for prompt tightening.

A.4 Confidence calibration

verdict	n	mean confidence
<code>entailed</code>	4,930	0.964
<code>not_entailed</code>	2,507	0.962
<code>entailed_direction_reversed</code>	552	0.926
<code>entailed_wrong_types</code>	3,014	0.912
<code>entailed_wrong_relation</code>	2,315	0.899
<code>partially_entailed</code>	2,151	0.847

The judge is uniformly high-confidence across verdicts (0.85–0.96), with `partially_entailed` the one verdict where confidence drops — consistent with it being the hedged/uncertain category. A fuller calibration would require token-log-probability scoring (available via the judge’s `--use-logprobs` mode).

A.5 NLI-judge agreement (open-domain run)

Of the **12,434** free-form triplets, **5,882** were judged entailed (47.3%). Among the entailed subset the two validators **fully agreed** on 2,760 (46.9%) and **partially agreed** on 3,122 (53.1%); the partial cases are where the NLI signal disagreed with the judge and the judge overrode it. The NLI label split

underlines why the judge is needed: at chunk level the cross-encoder said *entailment* 4,147 / *contradiction* 1,304 / *neutral* 431, while at span level it said *entailment* 4,495 / *contradiction* 429 / *neutral* 958 — the premise (chunk vs. span) swings the NLI verdict substantially.

Appendix B — Notes

Intermediate files and naming

In the file names, “canonical” means row-deduplicated with no IDs, and “canonicalized” means entity IDs minted.

file	rows	IDs	stage
<code>triplets.csv</code>	15,501	empty	raw constrained extraction
<code>triplets_verified.csv</code>	13,355	empty	after NLI + judge (kept or corrected)
<code>triplets_verified_canonical.csv</code>	12,400	empty	cleaned + three-phase dedup; adds <code>alt_snippet_ids</code> (714 rows) and <code>_type_fix</code> (531 rows)
<code>triplets_canonicalized.csv</code>	12,400	filled	<code>verified_canonical</code> + minted IDs; input to <code>build_rdf_kg.py</code>
<code>triplets_verified_canonicalized.csv</code>	13,355	filled	superseded: IDs minted before cleaning and dedup (still contains the 30 bad-source rows); not used downstream

`subject_id` / `object_id` and the `fkg:` id namespace

Canonicalisation is a separate downstream step, applied only once the full validated (and deduplicated) set is assembled: clustering entity mentions into stable identifiers needs to see the whole dataset at once, since assigning IDs incrementally risks the same entity receiving different IDs across batches. NLI/judge validation never assigned these IDs — it only checks truth against evidence, not identity — so the ID columns were empty until canonicalisation ran.

The canonicaliser emits instance IDs as `fkg:<TYPE>_NNNNN` (e.g. `fkg:F00_00001`). Note the `fkg:` prefix here is the **food-KG instance id namespace**, distinct from the `fkg ontology` namespace (`https://w3id.org/fkg/ontology#`). When the RDF is built, these are mapped to `urn:fflo:resource:<TYPE>_<NNNNN>` instance IRIs so they do not collide with ontology classes. If a pre-existing `subject_id / object_id` is already populated (e.g. by the NLI synthetic-id step), the canonicaliser writes its KG ids to separate `subject_canonical_id / object_canonical_id` columns instead of overwriting.

References

- [1] S. K. Gupta, L. Dey, P. P. Das, R. Jain. "Building FKG.in: A Knowledge Graph for Indian Food." 2024. <https://arxiv.org/pdf/2409.00830>
- [2] S. Haussmann et al. "FoodKG: A Semantics-Driven Knowledge Graph for Food Recommendation." ISWC 2019. <https://www.cs.rpi.edu/~zaki/PaperDir/ISWC19.pdf>
- [3] B. Zhang, H. Soh. "Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction." EMNLP 2024. <https://arxiv.org/pdf/2404.03868>
- [4] "KGValidator: A Framework for Automatic Validation of Knowledge Graph Construction." 2024. <https://arxiv.org/pdf/2404.15923>
- [5] "Traceable LLM-based Validation of Statements in Knowledge Graphs." 2025. <https://www.sciencedirect.com/science/article/abs/pii/S0306457325000706>
- [6] "Knowledge Graph and Large Language Model Synergy for Food Safety: Approaches and Perspectives." 2026. <https://www.sciencedirect.com/science/article/abs/pii/S0924224426000646>
- [7] "ODKE+: Ontology-Guided Open-Domain Knowledge Extraction with LLMs." 2025. <https://arxiv.org/html/2509.04696v1>
- [8] A. G. Regino, J. C. dos Reis. "Can LLMs be Knowledge Graph Curators for Validating Triple Insertions?" GenAIK 2025. <https://aclanthology.org/2025.genaik-1.10/>

Note: citation numbering here matches `kg_pipeline_method.md` and `article_draft_fflo.md` so the documents stay cross-referenceable.